

ASYNCSOURCE

REDUX TOOLKIT

REDUX-THUNK



1

ASYNCSOURCE

- An unencrypted, asynchronous, persistent, key-value storage system
- Allow persisting data offline in React Native apps
- Designed for small amounts of data. It's not efficient for large data sets

ASYNCSSTORAGE

- npm install **@react-native-async-storage/async-storage**
- Some methods
 - getItem()
 - setItem()
 - removeItem()
 - mergeItem()
 - clear()

Saving the data

```
static setItem(key: string, value: string,  
               [callback]: ?(error: ?Error) => void)
```

Parameters:

NAME	TYPE	REQUIRED	DESCRIPTION
key	string	Yes	Key of the item to set.
value	string	Yes	Value to set for the <code>key</code> .
callback	<code>?(error: ?Error) => void</code>	No	Function that will be called with any error.

Reading the data

```
static getItem(key: string, [callback]: ?(error: ?Error) => void)
```

Parameters:

NAME	TYPE	REQUIRED	DESCRIPTION
key	string	Yes	Key of the item to fetch.
callback	?(error: ?Error, result: ?string) => void	No	Function that will be called with a result if found or any error.

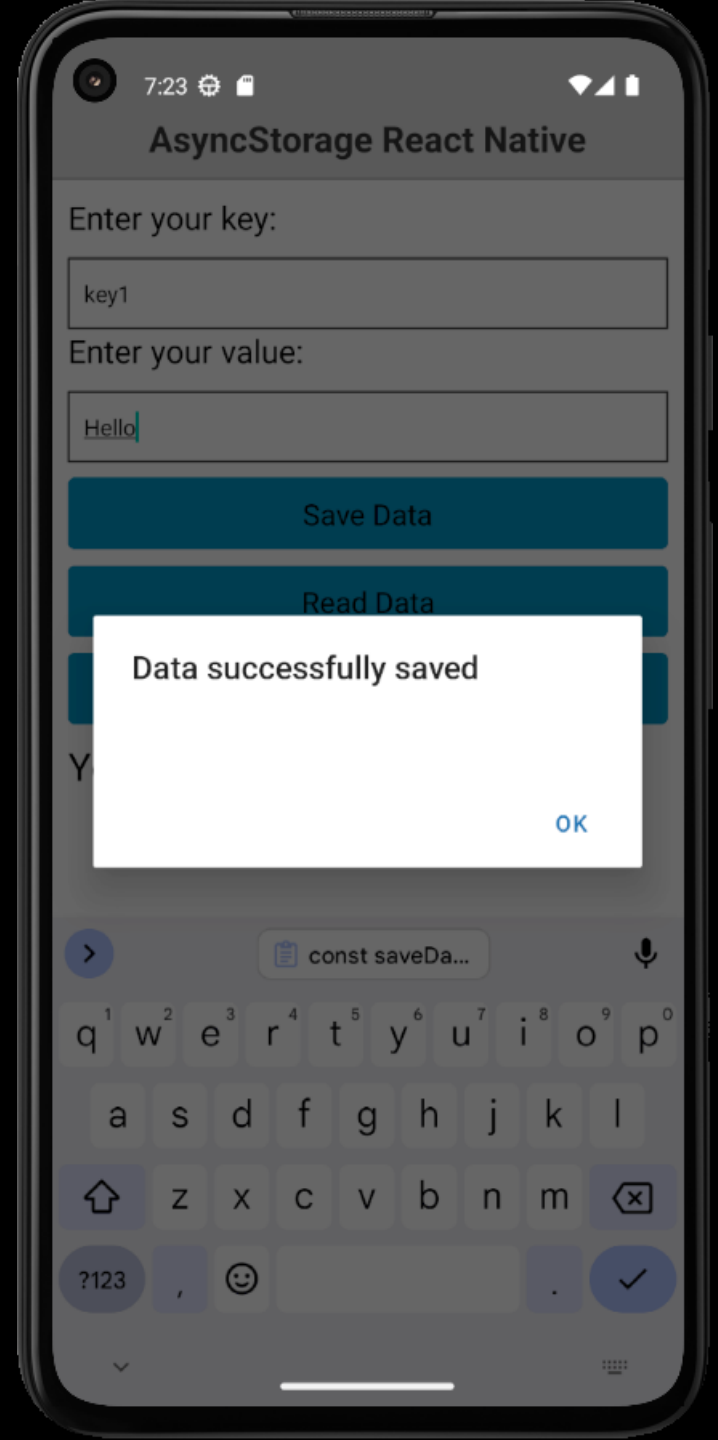
```

const saveData = async () =>
{
  try
  {
    await AsyncStorage.setItem(key, input)

    Alert.alert('Data successfully saved')
  } catch (e)
  {
    Alert.alert('Failed to save the data to the storage')
  }
}

const readData = async () =>
{
  try
  {
    const value = await AsyncStorage.getItem(key);
    if (value !== null)
    {
      setInput(value);
    } else
    {
      setInput('Data not found')
    }
  } catch (e)
  {
    Alert.alert('Failed to fetch the input from storage');
  }
};

```



2

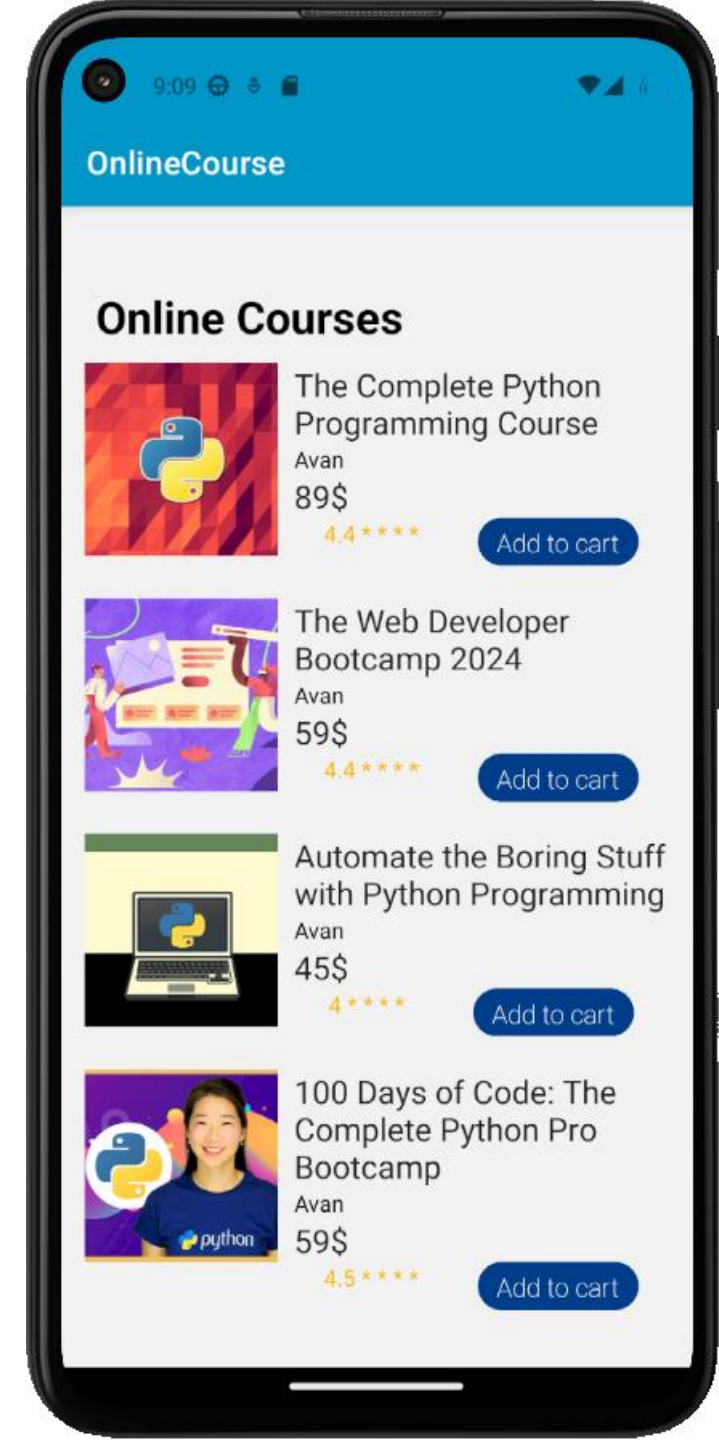
REDUX TOOLKIT

- Known as "RTK" for short
- Is intended to be the standard way to write Redux logic
- Best practices, simplifies most Redux tasks, prevents common mistakes, and makes it easier to write Redux applications

Redux Logic with Redux Toolkit

- Replace existing action and reducer with RTK's **createSlice**
- Replace the existing manual Redux store setup with Redux Toolkit's **configureStore**

Sample



Create a Redux State Slice

```
import { createSlice, RootState } from '@reduxjs/toolkit'

const initialState = [
  {
    id: 1,
    name: "The Complete Python Programming Course",
    img: "https://img-c.udemycdn.com/course/480x270/394676_ce3d_5.jpg",
    teacher: 'Avan', price: 89, rating: 4.4 },
  //...
]

export const courseSlice = createSlice({
  name: 'course',
  initialState,
  reducers: {
    getAllItems: (state, action) => {
      state
    }
  },
})

// Action creators are generated for each case reducer function
export const { getAllItems } = courseSlice.actions
export default courseSlice.reducer
export const getCourseState = (state) => state.course;
```

Add Slice Reducers to the Store

```
import { configureStore } from '@reduxjs/toolkit'
```

```
import courseReducer from './courseSlice'  
export const store = configureStore({  
  reducer: {  
    course: courseReducer  
  },  
})
```

Use Redux State and Actions in Components

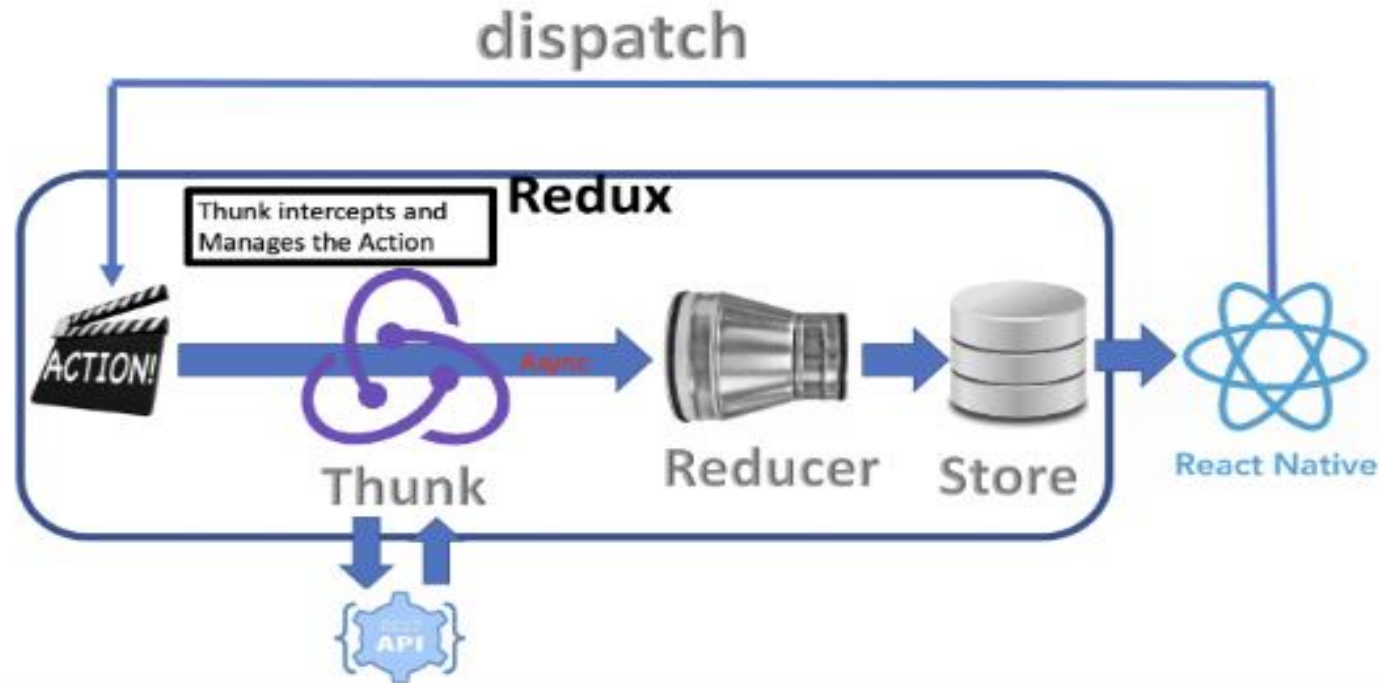

```
import { useSelector, useDispatch } from "react-redux";  
import { getCourseState, getAllItems } from '../reduxtoolkit/courseSlice';
```

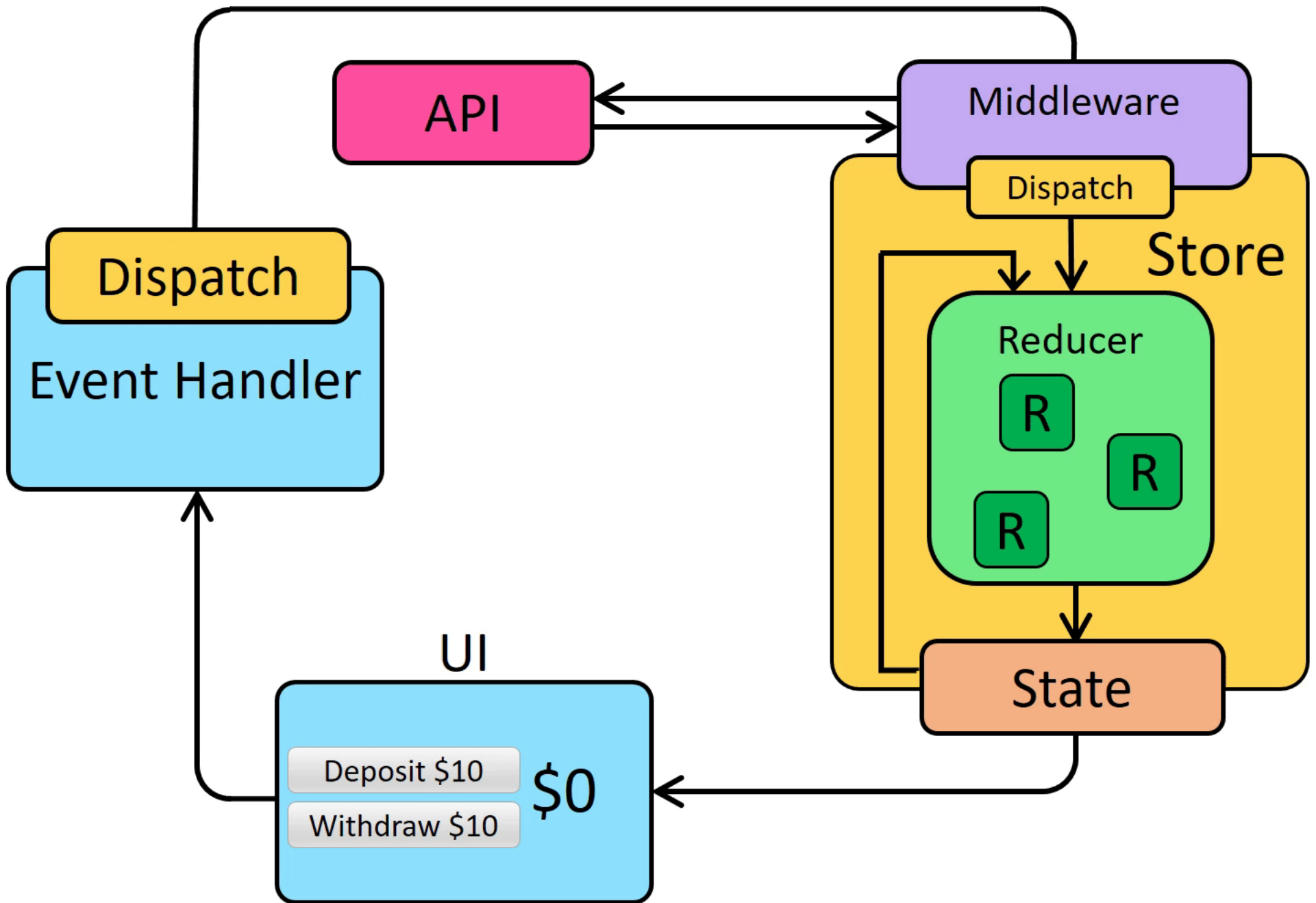
```
const OnlineCourses = ({navigation}) =>  
{  
  const [data, setData] = useState([]);  
  const dispatch = useDispatch();  
  const db = useSelector(rootState=> getCourseState(rootState))  
  
  useEffect(() => {  
    setData(db)  
  }, [data]);  
  
  // code here
```

3

REDUX-THUNK

- “Thunks” are a pattern of writing functions with logic inside that can interact with a Redux store's dispatch
- Using thunks requires the redux-thunk middleware to be added to the Redux store as part of its configuration

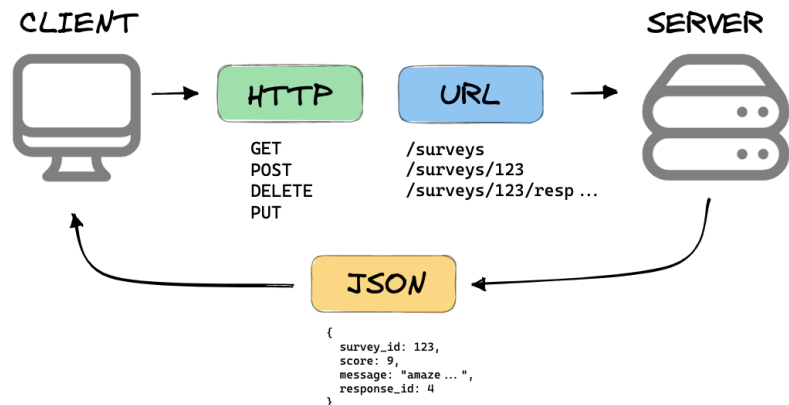




WHAT IS RESTFul API?

- Is the most common standard used between Clients who want to access information from the web from Servers (application or database)
- For test: <https://jsonplaceholder.typicode.com/>

WHAT IS A REST API?

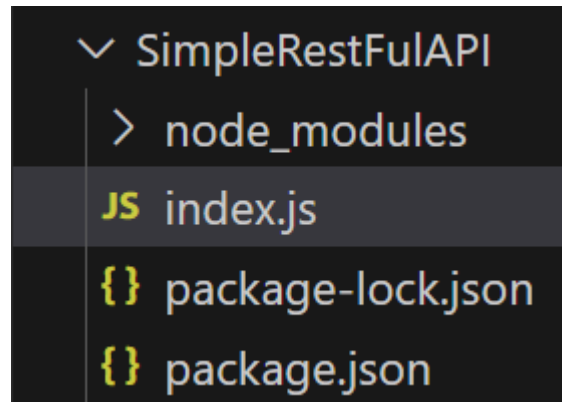


mannhowie.com

Source: <https://mannhowie.com/rest-api>

Building Simple RESTFul API

- Create new project
 - Cài đặt Nodejs tại link <https://nodejs.org/en>
 - Tạo thư mục SimpleResfulAPI\index.js
 - npm init
- Install library
 - npm install -g nodemon
 - npm install express -save
 - npm install body-parser -save
 - npm install cors



```
var express = require('express');
var cors = require('cors');
var bodyParser = require('body-parser');
var app = express();
app.use(bodyParser.urlencoded({ extended: false }));
app.use(bodyParser.json());
app.use(cors());
app.use(express.static('public'));
const PORT = process.env.PORT || 1234
app.listen(PORT, function () {
  console.log(`Simple ResfulAPI at: http://localhost:${PORT}`); });
const items =[{ id: 1,
  name: "The Complete Python Programming Course",
  img: "https://img-c.udemycdn.com/course/480x270/394676_ce3d_5.jpg",
  teacher: 'Avan',price: 89,rating: 4.4 },
{id: 2,
  name: "The Web Developer Bootcamp 2024",
  img: "https://img-c.udemycdn.com/course/750x422/625204_436a_3.jpg",
  teacher: 'Avan', price: 59,rating: 4.4
},]

app.get('/courses', async (req, res) => {
  return res.status(200).json(items)
})
```

GET

<http://localhost:1234/courses>



← → ↻ 🏠 ⚠ Not secure 192.168.1.3:1234/courses

Pretty-print ☒

```
[
  {
    "id": 1,
    "name": "The Complete Python Programming Course",
    "img": "https://img-c.udemycdn.com/course/480x270/394676_ce3d_5.jpg",
    "teacher": "Avan",
    "price": 89,
    "rating": 4.4
  },
  {
    "id": 2,
    "name": "The Web Developer Bootcamp 2024",
    "img": "https://img-c.udemycdn.com/course/750x422/625204_436a_3.jpg",
    "teacher": "Avan",
    "price": 59,
    "rating": 4.4
  }
]
```


Fetch API

- Allow loading resources from a **remote URL**
- Fetch API
 - Making request
 - Handling response

Making Request

- How to make GET request ?

```
fetch('https://mywebsite.com/mydata.json');
```

- How to make POST request

```
fetch('https://mywebsite.com/endpoint/', {  
  method: 'POST',  
  headers: {  
    Accept: 'application/json',  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    firstParam: 'yourValue',  
    secondParam: 'yourOtherValue'  
  })  
});
```

Handling the Response

use Promise

```
const getMoviesFromApi = () => {  
  return fetch('https://reactnative.dev/movies.json')  
    .then((response) => response.json())  
    .then((json) => {  
      return json.movies;  
    })  
    .catch((error) => {  
      console.error(error);  
    });  
};
```

Use async / await

```
const getMoviesFromApiAsync = async () => {  
  try {  
    const response = await fetch(  
      'https://reactnative.dev/movies.json'  
    );  
    const json = await response.json();  
    return json.movies;  
  } catch (error) {  
    console.error(error);  
  }  
};
```

Redux & Redux-Thunk

- npm install **redux**
- npm install **react-redux**
- npm install **@reduxjs/toolkit**
- npm install **@react-native-async-storage/async-storage**
- Steps
 1. createSlice
 2. createReducer
 - 3. createAsyncThunk**
 4. Store

```
import {createSlice,RootState } from '@reduxjs/toolkit'

const initialState = {
  items: []
}

export const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    addItem: (state,action) => {
      if ( state.items.find((item)=> item.id===action.payload.id)===undefined)
        state.items.push(action.payload)
    },
    removeItem: (state,action) => {
      state.items=state.items.filter((item)=> item.id !== action.payload)
    },
  }
})

// Action creators are generated for each case reducer function
export const { addItem,removeItem } = cartSlice.actions
export default cartSlice.reducer
export const getCartState = (state) => state.cart;
```

```
import { createAsyncThunk, createSlice, RootState } from '@reduxjs/toolkit'
const initialState={
  courses: [] }
export const fetchAllItems = createAsyncThunk('course/fetchAllItems',
  async () => {
    const data = await fetch('http://192.168.1.3:1234/courses')
      .then(response=> response.json())

    console.log(data)
    return data
  })
export const courseSlice = createSlice({
  name: 'course',
  initialState,
  reducers: {},
  extraReducers: (builder) => {
    builder.addCase(fetchAllItems.fulfilled, (state, action) => {
      state.courses=action.payload
      console.log('done', [state,action])
    })
    builder.addCase(fetchAllItems.rejected, (state, action) => {
      state.courses= []
      console.log('rejected', [state,action])
    })
  },
})
// Action creators are generated for each case reducer function
export const { getAllItems } = courseSlice.actions
export default courseSlice.reducer
export const getCourseState = (state) => state.course;
```



```
import { configureStore } from '@reduxjs/toolkit'
import cartReducer from './cartSlice'
import courseReducer from './courseSlice'
export const store = configureStore({
  reducer: {
    cart: cartReducer,
    course: courseReducer
  },
})
```

```
import { Provider } from "react-redux";
import {store} from './reduxtoolkit/myStore'
import CourseStack from './routes/CourseStack';
import {NavigationContainer} from '@react-navigation/native'

export default function App() {
  return (
    <Provider store={store} style={styles.container}>
      <NavigationContainer>
        <CourseStack/>
      </NavigationContainer>

    </Provider>

  )
}
```